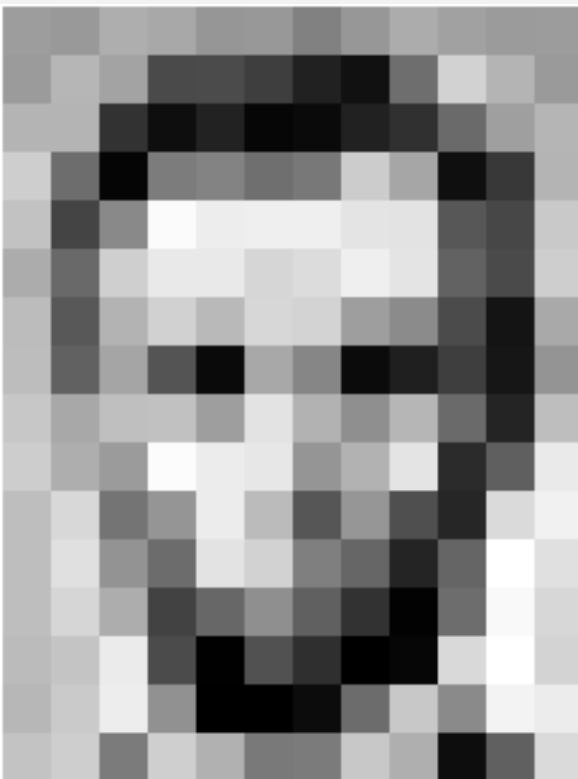


FUNDAMENTALS OF IMAGE PROCESSING

WHAT IS A GRAY IMAGE



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	105	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

HOW TO REPRESENT A MATRIX IN PYTHON

- A 2D matrix can be represented with Python Lists

```
[[1,0,1],[0,0,0],[1,1,1]]
```

- or easier to see/present/understand

```
[[1,0,1],
```

```
[0,0,0],
```

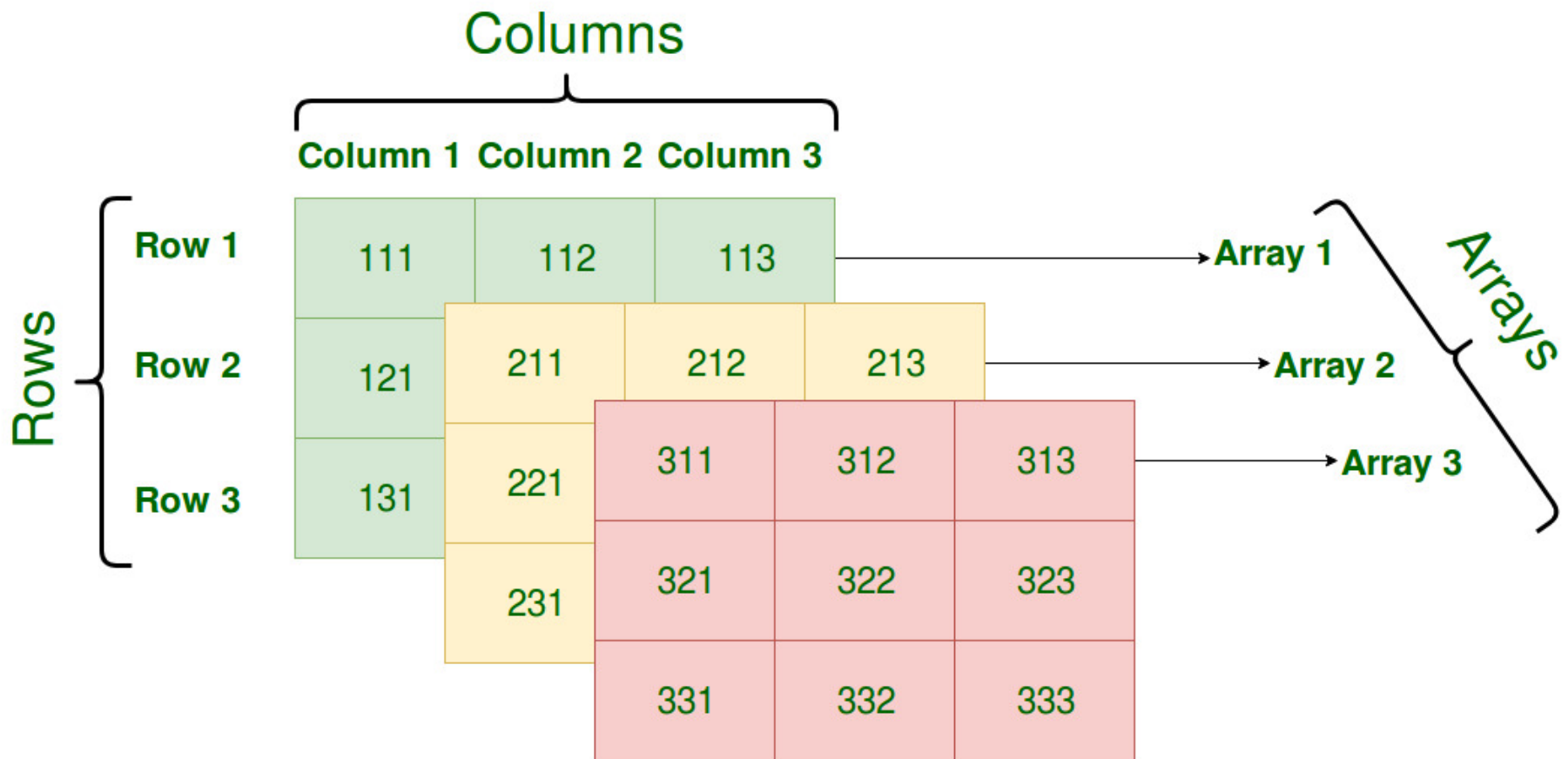
```
[1,1,1]]
```

- Lists need too much working storage and can contain different types (str,int,object,etc)
- Working/programming with lists is relatively slow

HOW TO REPRESENT A MATRIX IN PYTHON

- A way around the speed and efficiency problem with lists is to use list-like objects that only one variable type. Numpy arrays are such an object and solve this problem
- For images, arrays are integer-type, corresponding to pixel positions
- Numpy = Numeric Python
- That's why digital images are normally Numpy Objects / Numpy Arrays

HOW TO REPRESENT A 3D IMAGE



HOW TO REPRESENT A 3D MATRIX

- To represent colors, there are different color systems
- See https://www.w3schools.com/colors/colors_picker.asp
- Mix Colors using the color converter
 - https://www.w3schools.com/colors/colors_converter.asp
- The most common system is the RGB system
- RGB: Red / Green / Blue
- Every color can be mixed with these three colors

HOW TO REPRESENT A 3D MATRIX

- Like when we were looking at the nested dictionaries of the NCBI API, the number of „[“ on the left side of the first list item gives the dimension of the image (careful, the first list item can be a list!)
- As in the 2D images, a 3D image is normally represented as a Numpy Array or Numpy Object
- Each Pixel contains 3 Color Layers (Red,Green,Blue)
- Summary: A Numpy Object needs less storage than a list, and contains one type in an array which is enough for representing images, and it's quicker for programming

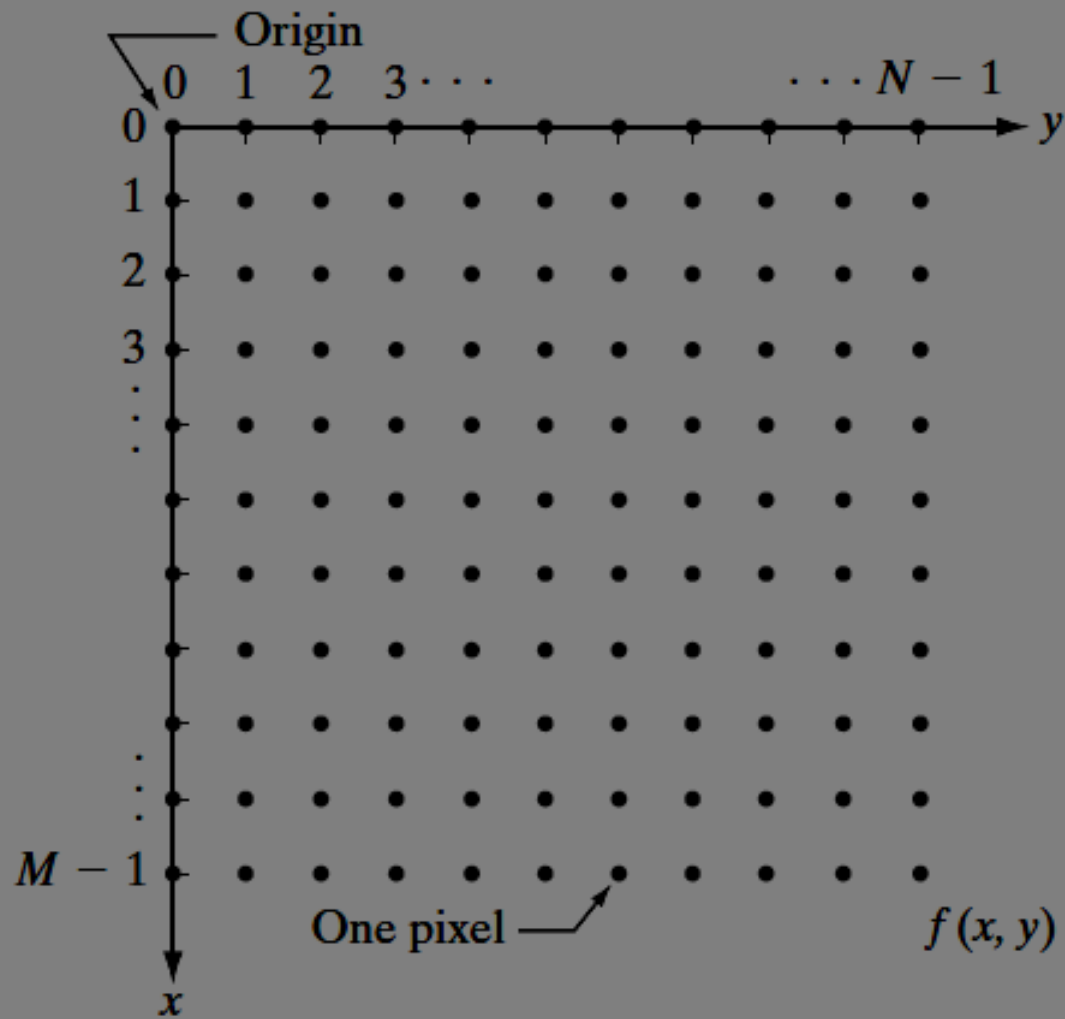
HOW TO REPRESENT A 3D MATRIX

- We can use a 3D List (example 8 bit integers)
- `[[[255,110,0],[100,200,250],[150,20,140]],`
- `[[100,200,50],[150,100,30],[120,150,30]],`
- `[[20,101,201],[10,110,90],[20,100,255]]]`
- 0 → max 255 are 8 bit integers ($2^8 - 1$) or
- $1*2^0 + 1*2^1 + 1*2^3 + 1*2^4 + 1*2^5 + 1*2^6 + 1*2^7$
- Max number with 2 digits and base 10 ($9*10^0 + 9*10^1 = 99$).
Computer → base 2 (0 or 1)
- Each position in the 2D Matrix is now a 1D matrix with 3 values (R-G-B) → a 3D Matrix

IMAGES CAN BE MORE THAN 3D

- Images can be 3D, 4D, 5D or maybe more
- Each dimension higher than the third contains information about the image
- OME-TIFF images are 5D images and one dimension is in XML format containing Information about the Image
- One critical Information ist the pixel size to know the real size of the Image
- ZEISS has images in 4D format (4th dimension is the time lapse)

MATRICES AND MATHEMATICS



MATRICES AND MATHEMATICS

$$f(x, y) = \begin{bmatrix} f(0, 0) & f(0, 1) & \cdots & f(0, N - 1) \\ f(1, 0) & f(1, 1) & \cdots & f(1, N - 1) \\ \vdots & \vdots & & \vdots \\ f(M - 1, 0) & f(M - 1, 1) & \cdots & f(M - 1, N - 1) \end{bmatrix}$$

$$\mathbf{A} = \begin{bmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,N-1} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,N-1} \\ \vdots & \vdots & & \vdots \\ a_{M-1,0} & a_{M-1,1} & \cdots & a_{M-1,N-1} \end{bmatrix}$$

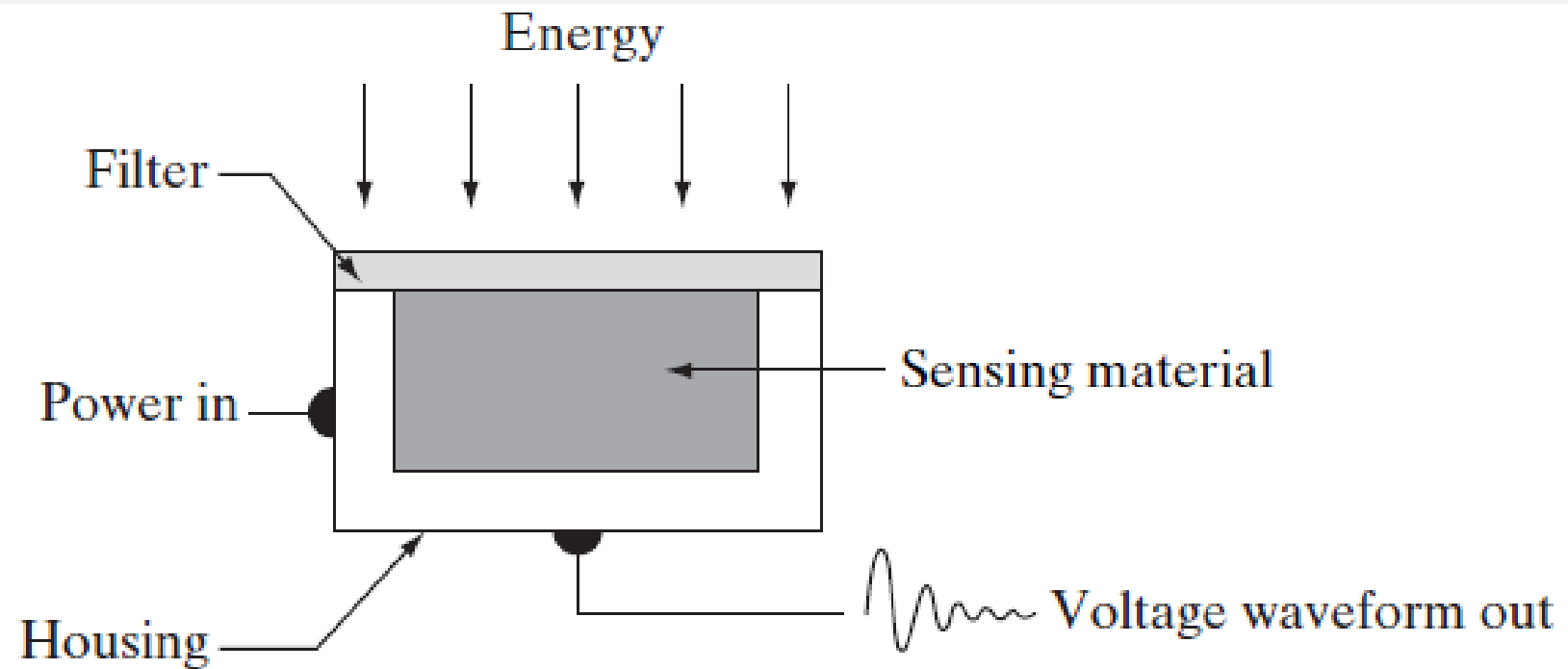
MANIPULATING IMAGES

- Because images are Numpy Arrays, we can modify every digital image when we modify the array matrix of the image
- Every mathematical operation on matrices can therefore be done on digital images
- Layers of 3D images can be sliced to create 2D Images
- A 2D grey image can be created by slicing the RED/GREEN/BLUE Layer of the Image

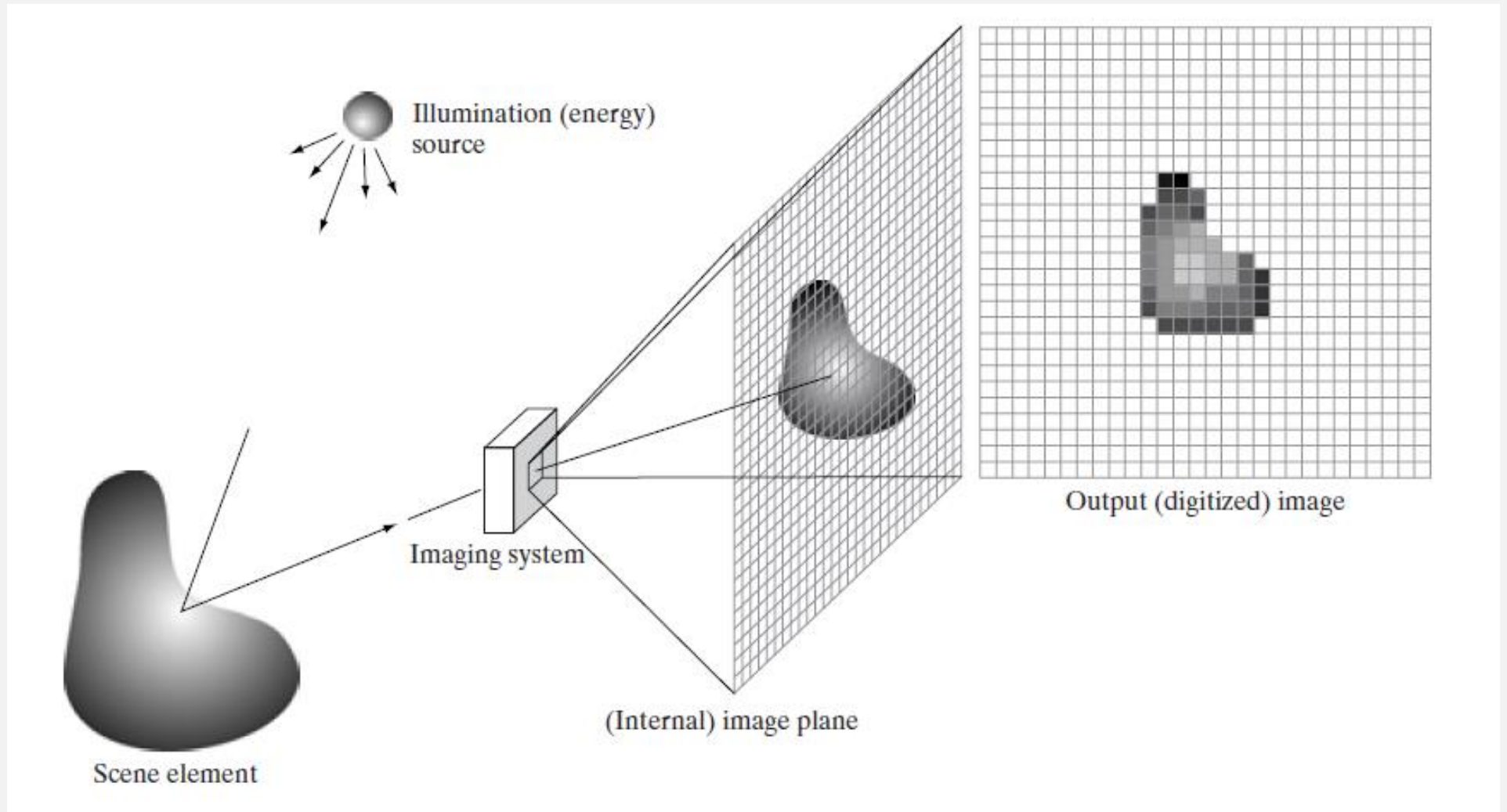
HOW TO MAKE DIGITAL IMAGES

- Images can be captured using single sensing elements
- A familiar sensor of this type is the photodiode
- A photodiode is constructed of silicon materials and its output is a voltage proportional to light intensity
- A geometry used more frequently than single sensors is an in-line sensor strip
- This arrangement is used in most flatbed scanners

HOW TO MAKE DIGITAL IMAGES



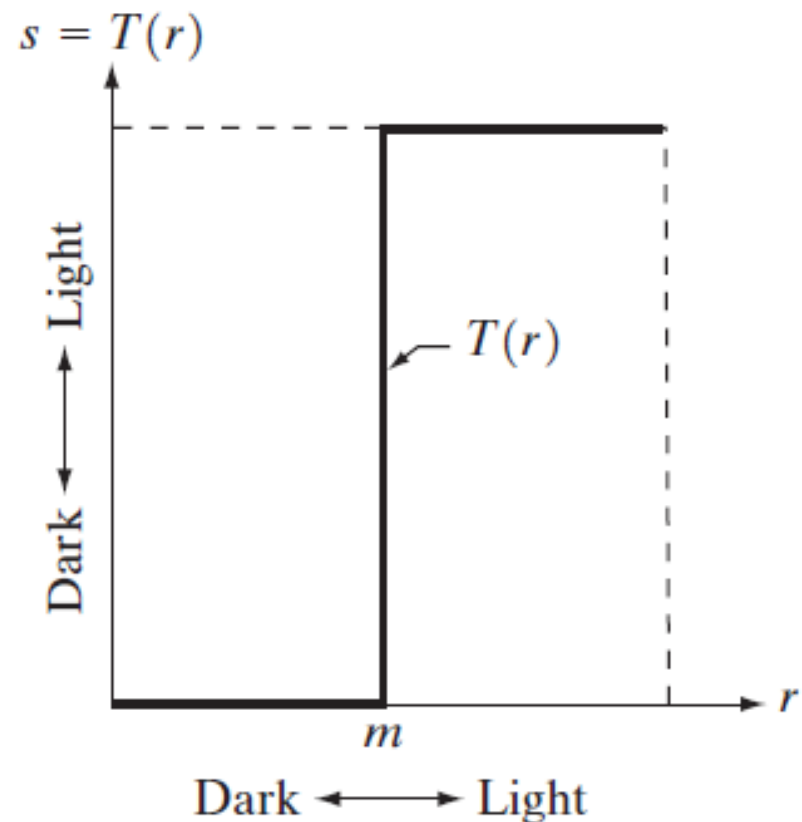
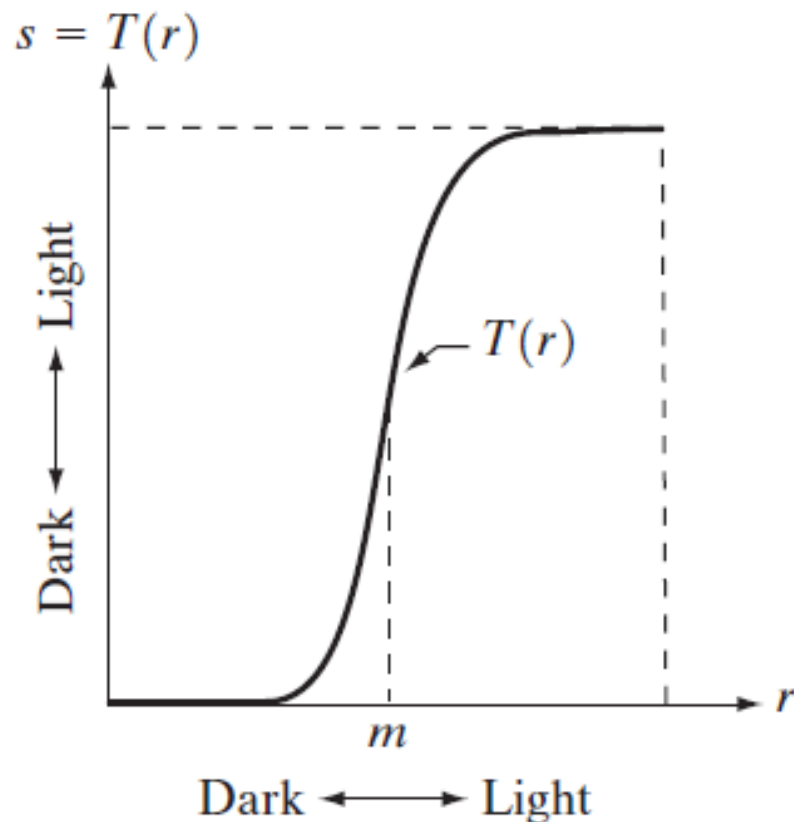
HOW TO MAKE DIGITAL IMAGES



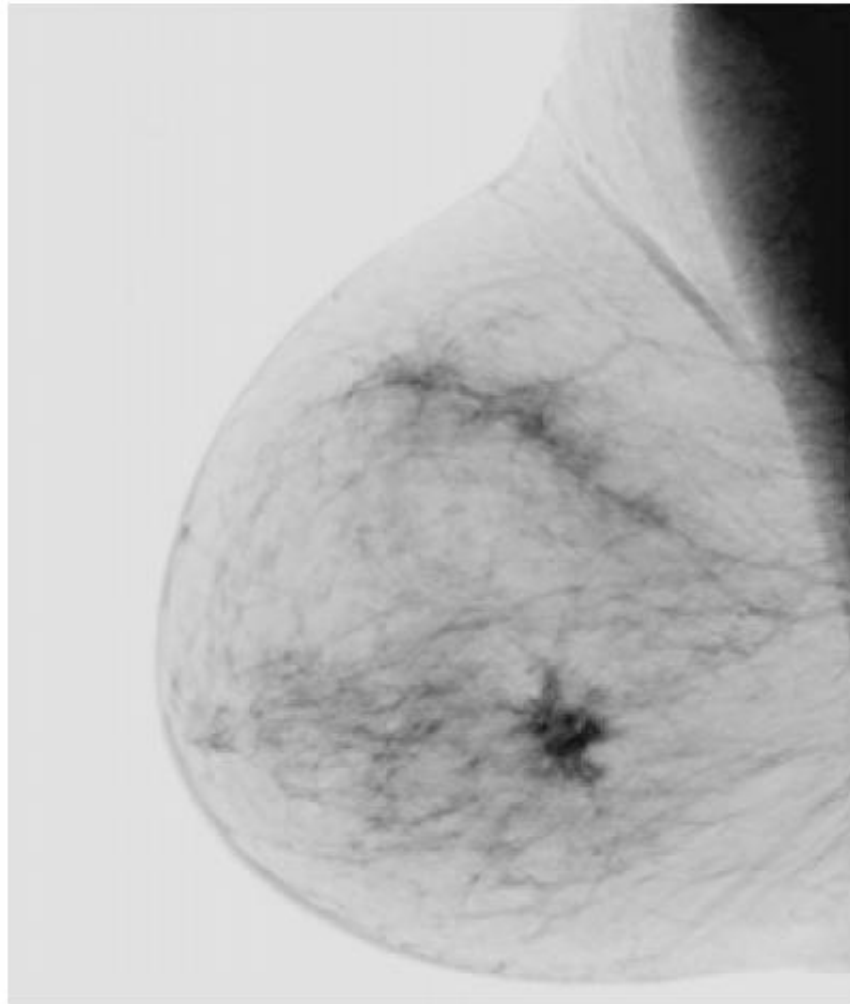
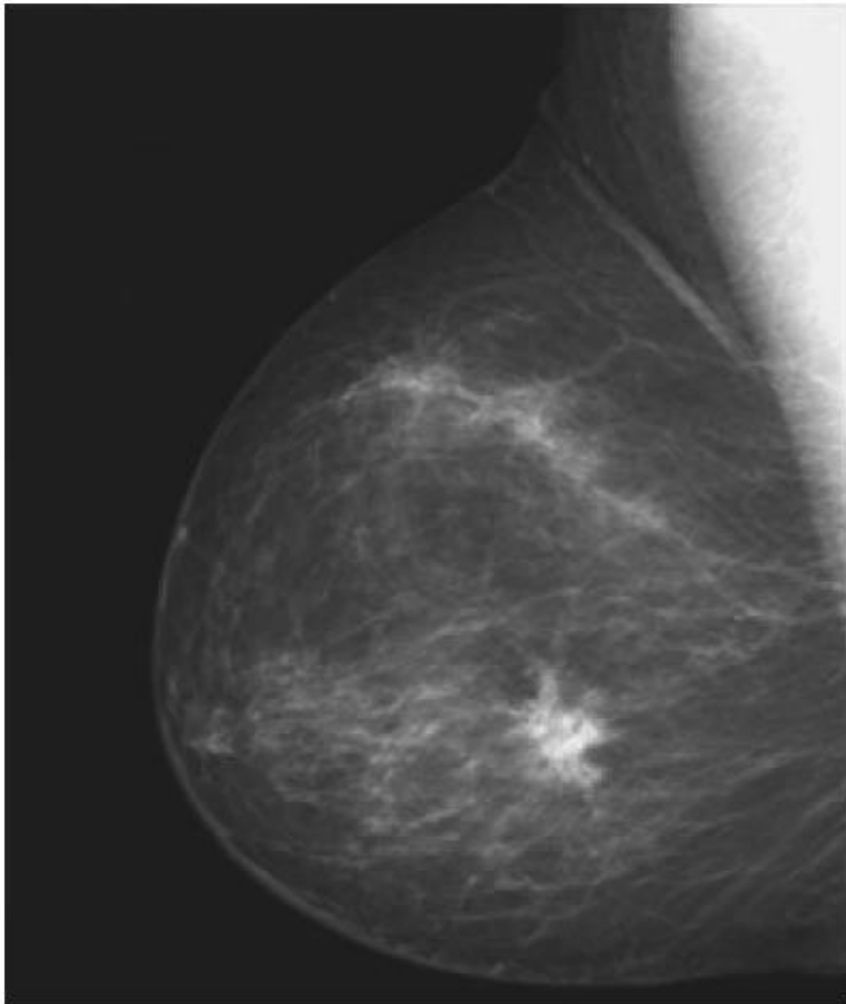
SOME SIMPLE OPERATIONS ON IMAGES / MATRICES

- Single Pixel Operations
- The simplest operation to perform on a digital image
$$s(z) = T(z) \quad \text{or} \quad g(x,y) = T[f(x,y)]$$
- z or $f(x,y)$ = Intensity of a pixel in the original image
- T = Transformation function
- $s(\text{pixel})$ = Is the mapped intensity of the corresponding pixel in the processed image.

SIMPLE OPERATIONS ON IMAGES: LEVEL TRANSFORMATION FUNCTIONS FOR CONTRAST ENHANCEMENT



SOME BASIC LEVEL GRAY TRANSFORMATIONS: REVERSING THE INTENSITY LEVELS OF THE IMAGE



SOME SIMPLE OPERATIONS ON IMAGES; NEIGHBORHOOD OPERATIONS

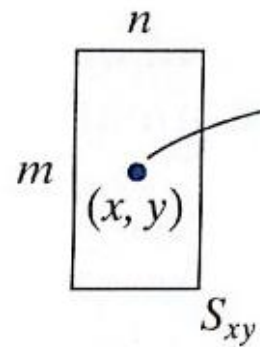


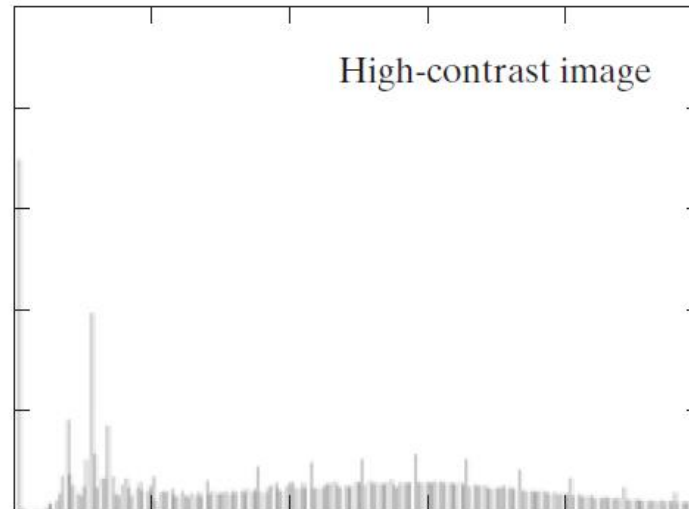
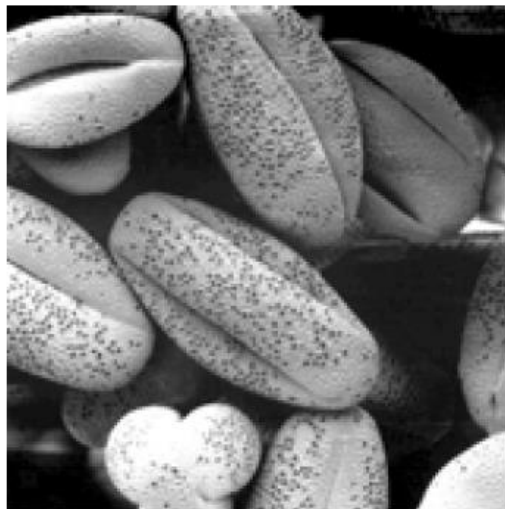
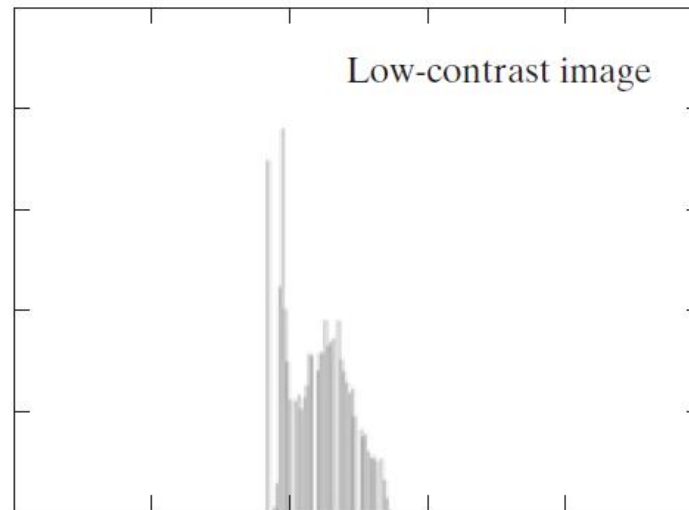
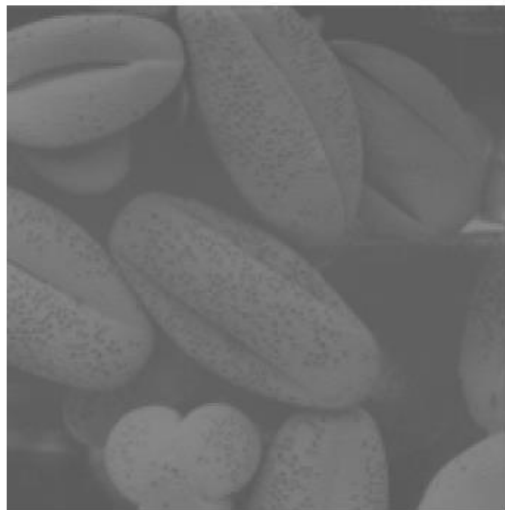
Image f



The value of this pixel
is the average value of the
pixels in S_{xy}

Image g

SOME OPERATIONS ON IMAGES



SOME OPERATIONS ON IMAGES

Let's unpack what those histograms meant:

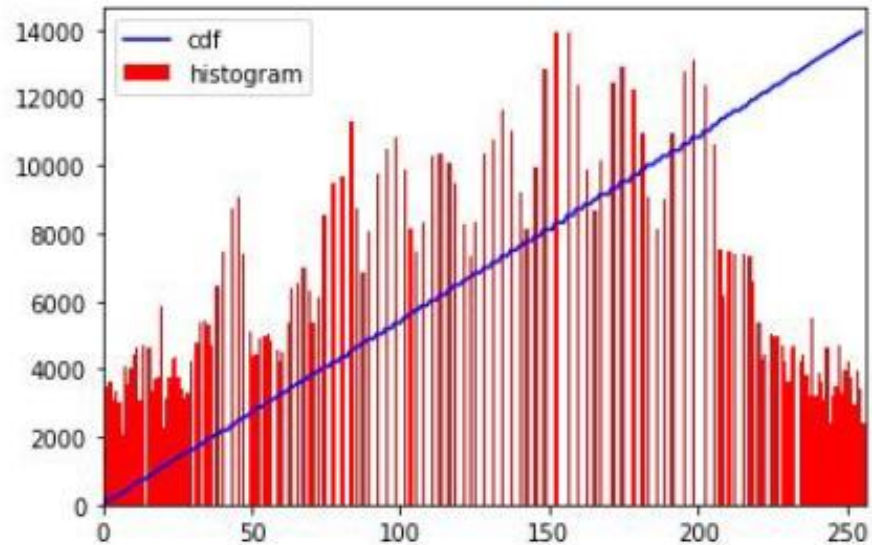
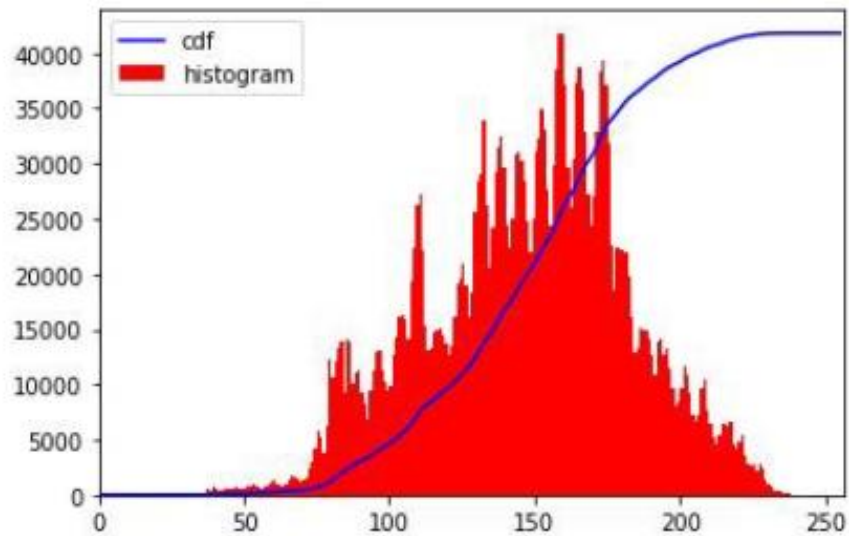
- Note: The histogram of dark or light or low-contrast images is narrow. Images with wide histograms deliver high-contrast images
- A histogram of an image is like a bar plot for the intensities of pixels of an image, but with ranges
- The bins of a bar plot gives the number of bars for intensities which are gathered together

SOME OPERATIONS ON IMAGES: HISTOGRAM EQUALISATION

(Don't worry, this will be represented visually in the next slide

- Histogram Equalization is a computer image processing technique used to improve contrast in images. It accomplishes this by effectively spreading out the most frequent intensity values, i.e. stretching out the intensity range of the image. This method usually increases the global contrast of images when its usable data is represented by close contrast values. This allows for areas of lower local contrast to gain a higher contrast.
- <https://towardsdatascience.com/histogram-equalization-5d1013626e64>

HISTOGRAM EQUALISATION: CDF (CUMULATIVE DISTRIBUTION FUNCTION)



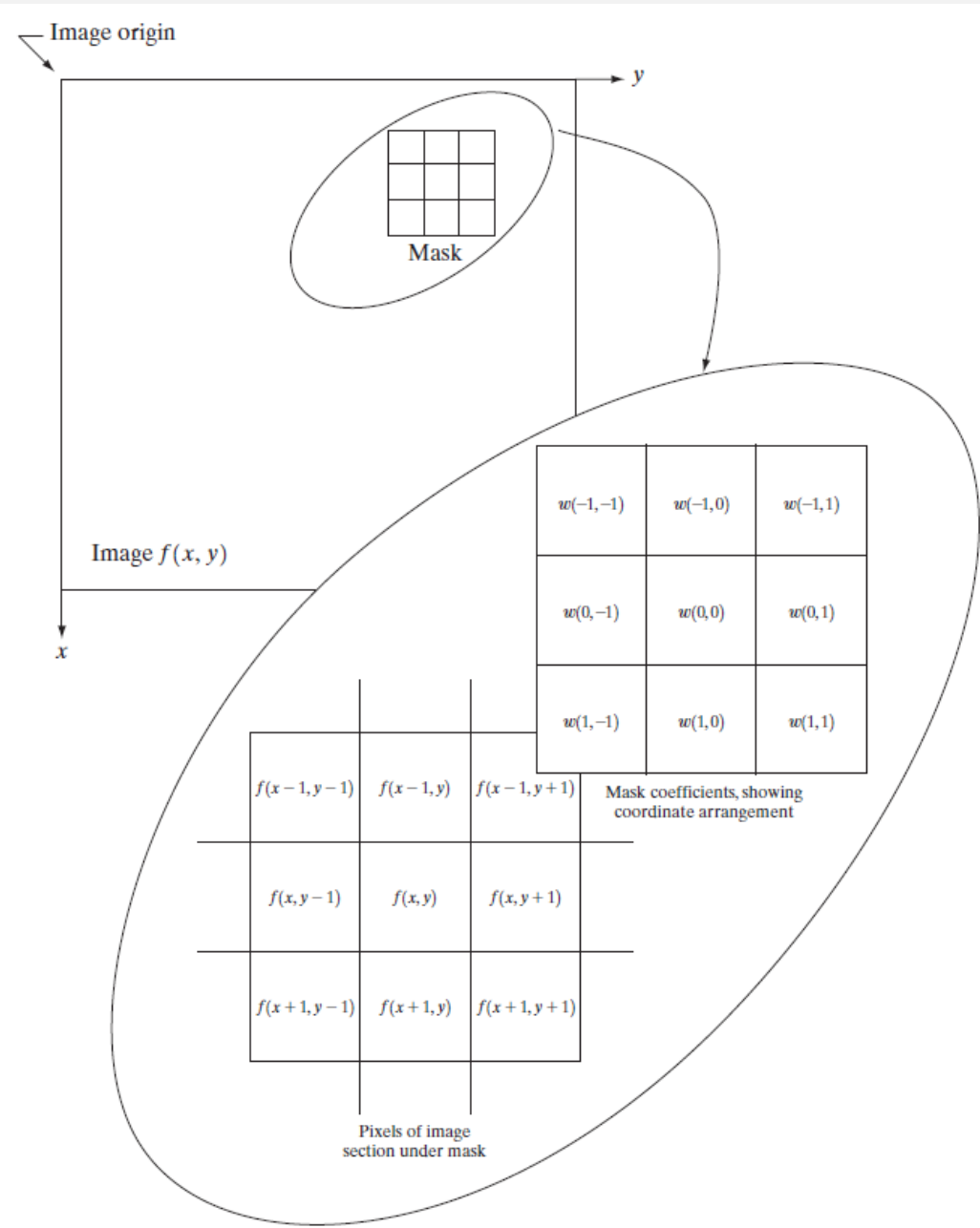
Comparison of Original and Histogram Equalized Histograms

SPATIAL FILTERING

- The process consists simply of moving the filter mask from point to point in an image. At each point (x, y) , the *response* of the filter at that point is calculated using a predefined relationship
- For *linear* spatial filtering, the response is given by a sum of products of the filter coefficients and the corresponding image pixels in the area spanned by the filter mask

$$R = w(-1, -1)f(x - 1, y - 1) + w(-1, 0)f(x - 1, y) + \cdots \\ + w(0, 0)f(x, y) + \cdots + w(1, 0)f(x + 1, y) + w(1, 1)f(x + 1, y + 1),$$

SPATIAL FILTERING



MEAN FILTERS

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$K = \frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

```
box_kernel = [[1 / 9, 1 / 9, 1 / 9],  
              [1 / 9, 1 / 9, 1 / 9],  
              [1 / 9, 1 / 9, 1 / 9]]
```

MEAN FILTERS

- What is the problem with these simple kernels?
- Do you have other suggestions?
- Imagine running a mean filter on salary of customers in a bar that you have 2D top-down view measurements of. What happens to your output when Bill Gates is inside?

GAUSSIAN KERNEL / MASK

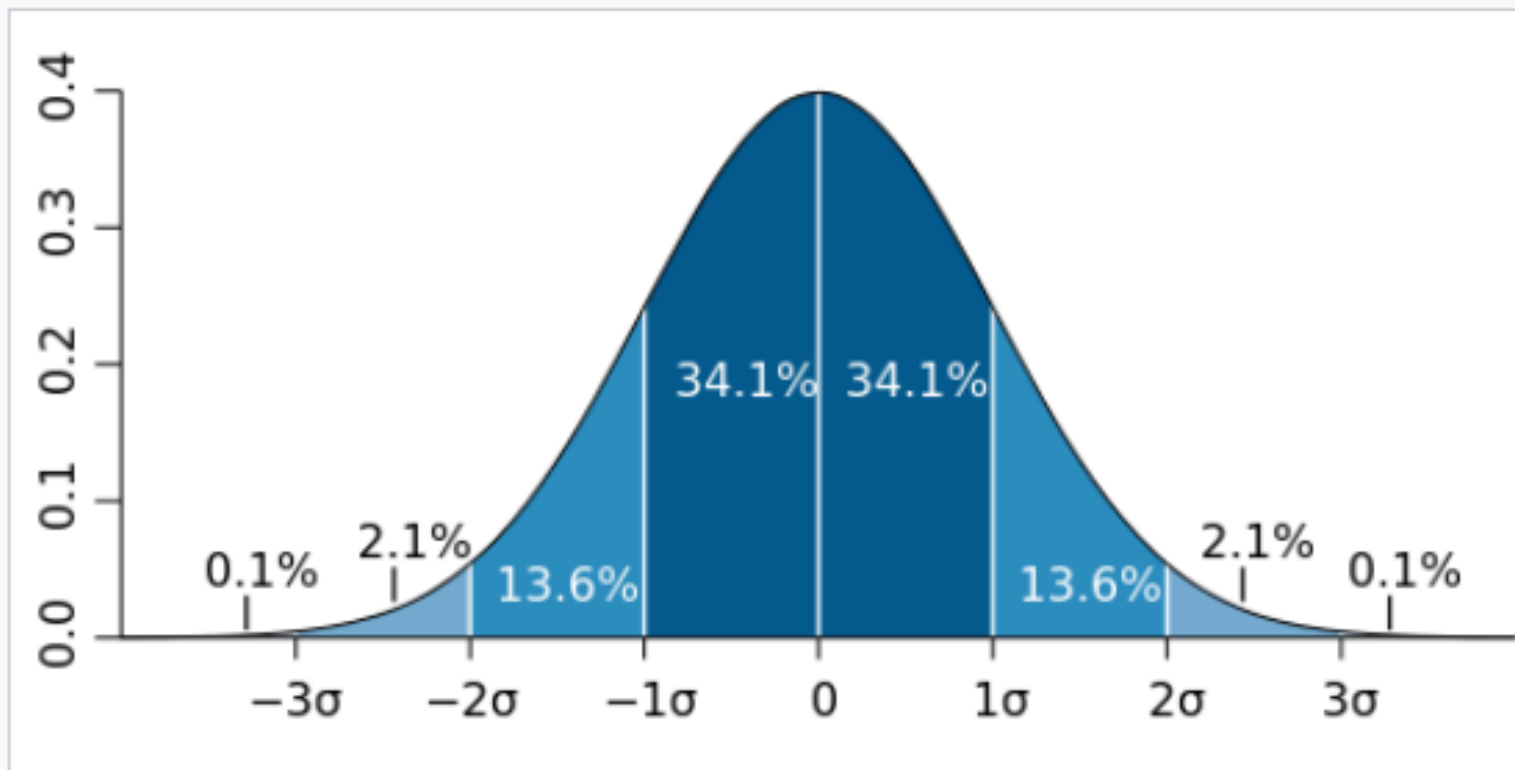
$$G(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}$$

- Gauss 1D

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

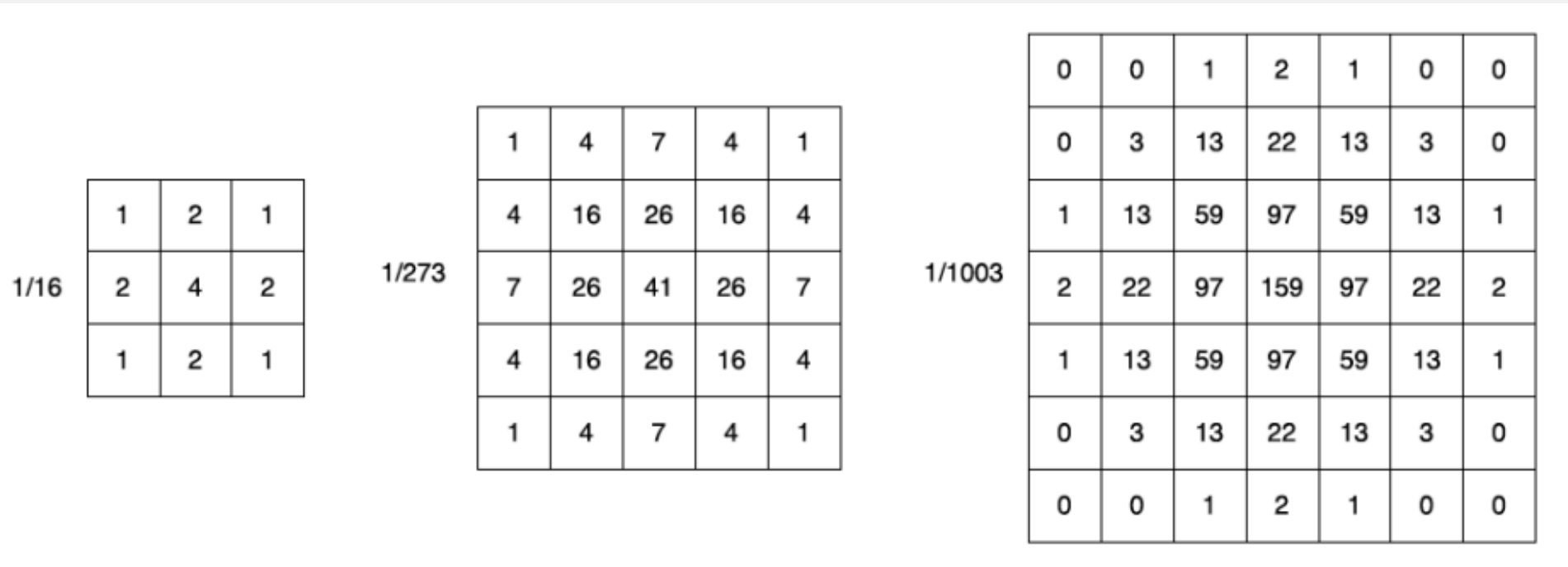
- Gauss 2D

GAUSSIAN KERNEL / MASK



For the normal distribution, the values less than one standard deviation away from the mean account for 68.27% of the set; while two standard deviations from the mean account for 95.45%; and three standard deviations account for 99.73%.

GAUSSIAN KERNEL / FILTER



- Note: The sum of values should be 1, so the result is normalized

MEDIAN FILTER

3 x 3 neighborhood

38	17	91
14	66	5
29	111	42

Sorting

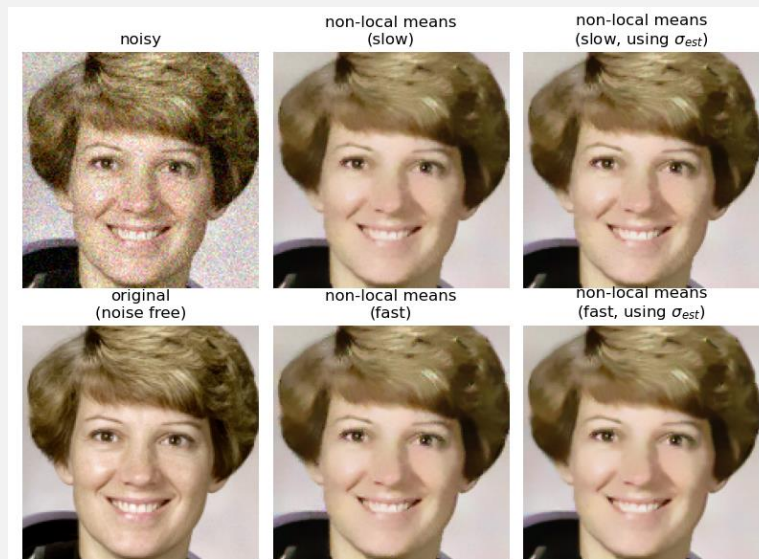
38	17	91
14	38	5
29	111	42

Replacing

5	14	17	29	38	42	66	91	111
---	----	----	----	----	----	----	----	-----

NON LOCAL MEANS FILTER

- The non-local means algorithm replaces the value of a pixel by an average of a selection of other pixels values: small patches centered on the other pixels are compared to the patch centered on the pixel of interest, and the average is performed only for pixels that have patches close to the current patch. As a result, this algorithm can restore well textures, that would be blurred by other denoising algorithm



Non-local means run on the altered image “noisy” with original for control comparison.

Source: scikit image

MAIN INFORMATION SOURCE

- Digital Image Processing, Global Edition
- ISBN-10 : 1292223049
- ISBN-13 : 978-1292223049